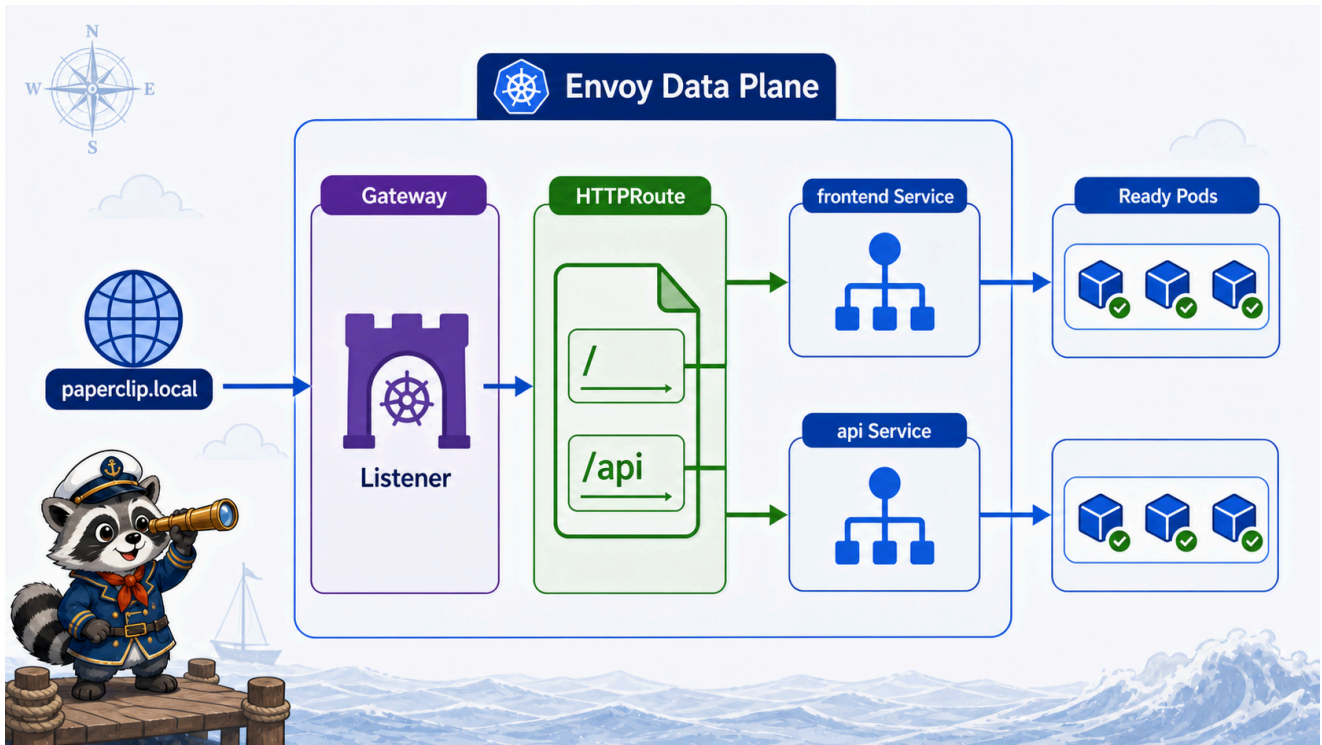


4교시: HTTPRoute 작성



수업 목표

- Gateway와 HTTPRoute를 이용해 host/path routing을 작성한다.
- /는 frontend, /api는 api로 routing한다.
- curl과 browser 관점에서 Host header 확인 방법을 익힌다.

Gateway manifest

```
cat week4/day2/labs/traffic-routing/gateway.yaml
```

핵심:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: paperclip-gateway
  namespace: week4
spec:
  gatewayClassName: envoy-gateway
  listeners:
```

```
- name: http
  protocol: HTTP
  port: 80
  hostname: paperclip.local
```

해석:

필드	의미
gatewayClassName	Envoy Gateway가 처리할 class
listeners[].protocol	HTTP traffic을 받음
listeners[].port	80번 listener
listeners[].hostname	paperclip.local host 기준
allowedRoutes.namespaces.from: Same	같은 namespace의 Route만 attach 허용

Gateway는 "어떤 문을 열 것인가"를 선언한다. 어디로 보낼지는 HTTPRoute가 정한다.

HTTPRoute manifest

```
cat week4/day2/labs/traffic-routing/httproute.yaml
```

핵심:

```
spec:
  parentRefs:
    - name: paperclip-gateway
  hostnames:
    - paperclip.local
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api
      backendRefs:
        - name: api
          port: 80
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
```

```
- name: frontend
  port: 80
```

해석:

필드	의미
parentRefs	어떤 Gateway에 붙을지
hostnames	어떤 Host header와 매칭할지
matches.path	path 조건
backendRefs.name	연결할 Service
backendRefs.port	Service port

적용과 확인

```
kubectl apply -f week4/day2/labs/traffic-routing/gateway.yaml
kubectl apply -f week4/day2/labs/traffic-routing/httproute.yaml

kubectl -n week4 get gateway,httproute
kubectl -n week4 describe gateway paperclip-gateway
kubectl -n week4 describe httproute paperclip-routes
```

확인할 조건:

리소스	조건
Gateway	Accepted 또는 Programmed 계열 condition
HTTPRoute	Accepted, ResolvedRefs
Service	frontend, api 존재
EndpointSlice	Ready endpoint 존재

HTTPRoute가 보인다고 traffic이 성공한다는 뜻은 아니다. Route가 Gateway에 attach되고 backendRefs가 Service로 해석되어야 한다.

backend Service 확인

```
kubectl -n week4 get svc,endpointslice -l kubernetes.io/service-name=api
kubectl -n week4 get svc,endpointslice -l kubernetes.io/service-name=frontend
```

EndpointSlice label이 환경에 따라 다르게 보이면 넓게 본다.

```
kubectl -n week4 get endpointslice
kubectl -n week4 get svc,endpoints
```

curl로 확인

port-forward 터미널:

```
kubectl get svc -A | grep envoy
kubectl -n envoy-gateway-system port-forward svc/<envoy-service-name> 8080:80
```

확인 터미널:

```
curl -H "Host: paperclip.local" http://localhost:8080/
curl -H "Host: paperclip.local" http://localhost:8080/api
```

예상:

```
첫 번째 요청: Paperclip W4D2 Frontend HTML
두 번째 요청: {"service":"api","version":"v1","status":"ok"}
```

Host header가 중요하다. Gateway listener와 HTTPRoute host가 paperclip.local이므로 그냥 curl http://localhost:8080/api를 보내면 host가 맞지 않아 404가 날 수 있다.

browser 확인

브라우저에서 확인하려면 hosts 파일에 다음을 추가한다.

```
127.0.0.1 paperclip.local
```

그다음 port-forward가 켜진 상태에서 접속한다.

```
http://paperclip.local:8080/
http://paperclip.local:8080/api
```

PathPrefix

PathPrefix는 /api 아래 경로도 같은 backend로 보낼 수 있다는 뜻이다.

요청	backend
/api	api Service
/api/orders	api Service
/	frontend Service
/about	frontend Service

경로가 겹칠 때는 더 구체적인 path가 먼저 적용되어야 한다. 그래서 /api와 /를 함께 둘 수 있다.

Docker와의 비교

Docker reverse proxy에서는 대개 proxy 설정 파일에 직접 route를 적었다.

```
location /api {
    proxy_pass http://api:8080;
}
```

Kubernetes Gateway API에서는 이 의도를 HTTPRoute로 선언한다.

```
HTTPRoute /api
-> api Service port 80
-> EndpointSlice
-> api Pod 8080
```

이 차이를 이해해야 GitOps와 policy가 자연스럽다. routing 설정도 Kubernetes object이므로 Git에 저장하고, Argo CD로 sync하고, Kyverno로 정책을 걸 수 있다.

Evidence Note

```
# W4D2S4 HTTPRoute
- Gateway name:
- GatewayClass:
- HTTPRoute parentRefs:
- `/` backend:
- `/api` backend:
- curl `/` result:
- curl `/api` result:
```

한 줄 요약

Gateway는 traffic을 받을 문이고, HTTPRoute는 host/path 조건에 따라 Service backend를 고르는 routing 계약이다.